

EpiSCo-Guard: Evidence-Scoped Parallel Reasoning for Reliable CAR-bench Agents

Changjian Li

lichangjian85@gmail.com

Abstract

CAR-bench evaluates whether a tool-using assistant remains correct when requests are ambiguous, capabilities are absent, state changes over multiple turns, and domain policies constrain otherwise plausible actions. We present EpiSCo-Guard, a Track 2 agent that combines deterministic evidence and policy reasoning with adaptive parallel Cerebras rollouts. The central design rule is that completion must be proven by current, argument-matched tool evidence; a previous success cannot satisfy a new request. Candidate actions are canonicalized, verified against tool schemas and visible evidence, and selected by consensus before a deterministic A2A finalizer exposes only the chosen response and required token metrics. The promoted candidate solves all 129 tasks in our public-train validation run and improves a same-day targeted set from 1/5 for the frozen baseline to 5/5, while retaining the same full-set score. These results are development evidence and are not presented as a claim about the hidden set.

1 Motivation and Design Goals

CAR-bench exposes a gap between occasional task capability and reliable deployment behavior: an assistant must complete feasible requests, admit when a required capability is missing, and clarify material ambiguity [Kirmayr *et al.*, 2026]. Tool-using language agents can reason effectively with interleaved observations and actions [Yao *et al.*, 2023], but unconstrained generation also creates three recurring failure modes: fabricated completion, reuse of stale observations, and stochastic selection of an invalid action.

EpiSCo-Guard is designed around four goals. First, every claimed side effect must have a visible, successful tool result. Second, evidence must be scoped to the latest request and to the arguments that identify the requested object. Third, deterministic policy checks must remain authoritative even when an LLM candidate is confident. Fourth, Cerebras speed should buy independent reasoning diversity rather than deeper serial chains.

2 System Boundary

The official evaluator sends policy context, user messages, tool definitions, and tool results over A2A [A2A Protocol Working Group, 2025]. Our agent stores state per context identifier and returns either user-facing text or structured tool calls. It never executes CAR-bench tools, reads evaluator state, or observes task identifiers and reward signals. Private plans, candidate tables, verifier notes, and the evidence ledger never leave the agent boundary. This separation both respects the competition contract and prevents private reasoning text from influencing the simulated user.

3 Evidence-Scoped Control

3.1 Conversation and Evidence State

Incoming turns update a conversation store and an append-only evidence ledger. Each record carries its visible source, turn index, success state, and raw tool reference. When the agent emits a tool call, the call is retained privately until the evaluator returns a result; the result is then associated with the original arguments. This supplies provenance even when a tool result contains only a success flag.

Completion predicates use the latest relevant user turn as a temporal lower bound. For example, a successful destination replacement from an earlier turn cannot prove that a newly requested destination was installed. Navigation and email completion paths therefore require success after the latest request. Argument-keyed predicates additionally compare critical fields such as a new location identifier, route identifier, recipient, or POI search location.

3.2 Source-Scoped Observation Reuse

Observation tools can return structurally similar entities with different meanings. A charger found along a route must not satisfy a later request for a charger near a named location. POI evidence can therefore be filtered by tool source and by a critical argument subset. The current charger intent is derived from the latest relevant user language, separating local search from route search without named-task exceptions.

3.3 Policy and Tool Semantics

A visible-input policy graph records unresolved obligations, confirmations, and missing capabilities. Tool side effects are inferred from explicit metadata when available and from

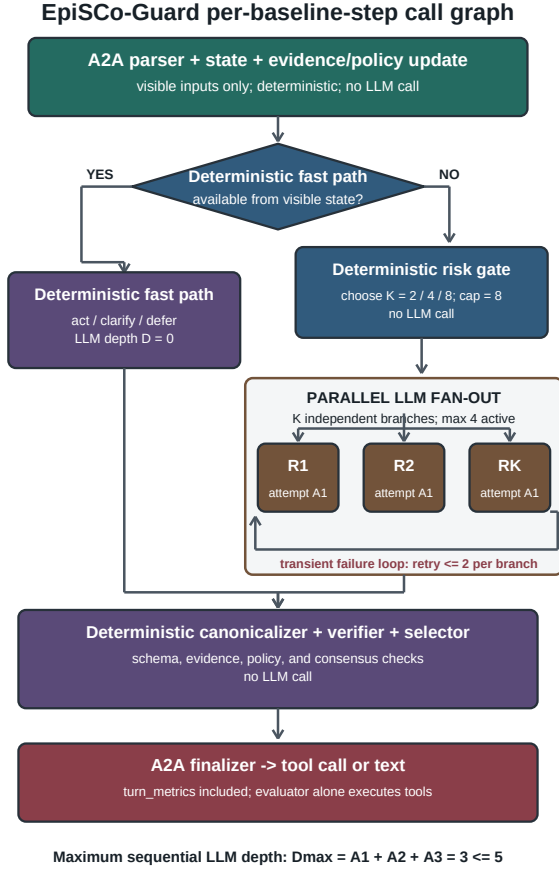


Figure 1: Auditable Track 2 call graph for each baseline-agent step. Fast paths use no LLM. The generative branch fans out independent rollouts; each branch has one initial call and at most two retries, so $D_{\max} = 3 \leq 5$.

generic leading action verbs otherwise. Read-only verb precedence prevents an incidental mutation word later in a tool name from misclassifying an observation tool. Generic mutation verbs such as `add` and `replace` receive mutation treatment.

The deterministic planner handles high-confidence policy and schema cases. Language parsing normalizes route endpoint order, navigation negation, discourse suffixes, and travel contrasts. These mechanisms are phrasing classes rather than exact test strings. A runtime literal scan found no challenge task IDs, generated proper-name exceptions, or exact challenge phrases.

4 Candidate Verification

Generated actions are parsed into a common candidate type: `act`, `clarify`, `defer`, or `respond-done`. The hard verifier rejects malformed JSON, unknown tools, schema-invalid arguments, unsupported claims, and unsafe side effects. Invalid clarify or defer candidates cannot cast minority vetoes. Selectable candidates are grouped by canonical action signature; the selector first maximizes group votes, then summed confidence,

then maximum member confidence. This prevents a single malformed but confident sample from defeating several equivalent valid actions.

The finalizer independently revalidates the selected action and renders the public A2A response. Clarifications are normalized as questions, and completion text is emitted only for a verified completion candidate.

4.1 Per-Turn Decision Procedure

Each assistant step follows a fixed control procedure. (1) The A2A parser normalizes the visible message, available tool schemas, and returned tool results. (2) The state layer associates newly returned evidence with pending outbound calls and updates policy obligations. (3) Deterministic handlers test for high-confidence grounded actions, explicit ambiguity, and unavailable capabilities. (4) If generative planning is needed, the risk model selects a rollout count and dispatches the samples in parallel. (5) Candidate JSON is normalized into the common action representation and checked independently by the hard verifier. (6) The selector aggregates only valid candidates by canonical action signature. (7) The finalizer performs the last contract check, records outbound tool-call provenance, and emits public A2A data with the aggregated turn metrics.

This ordering is important. The language model can propose an action, but it cannot create evidence that a tool ran, erase an unresolved policy obligation, or override a schema failure. Conversely, deterministic fast paths avoid an LLM call when the visible state already proves a unique safe response. The same pipeline handles base, hallucination, and disambiguation tasks; task category and task identifier are not runtime inputs.

5 Cerebras Fast-Reasoning Harness

The promoted full-mode runtime uses Cerebras-hosted `gpt-oss-120b` [OpenAI, 2025; Cerebras Systems, 2026] for candidate rollouts at medium reasoning effort. Planning, risk assessment, verification, selection, and finalization are deterministic and make no LLM calls. The submission scenario caps each completion at 2048 tokens, permits four active requests, and uses adaptive rollout counts of 2, 4, or 8 according to deterministic risk, capped at 8. Parallel candidates follow the self-consistency principle [Wang *et al.*, 2023], but selection is performed over executable action signatures rather than free-form answers.

Figure 1 shows every call-bearing branch. A deterministic fast path has sequential LLM depth zero. A generative path launches K independent rollouts; no candidate consumes another candidate’s output. Each rollout makes one initial provider call and, only after a transient failure, at most two retries. Thus the longest dependency chain is three LLM attempts, $D_{\max} = 3 \leq 5$. The four-request semaphore can schedule $K = 8$ in two waves, but this resource scheduling does not introduce an output dependency between the candidates.

Provider calls use bounded retries and jittered backoff. The provider adapter extracts prompt, completion, and reasoning-token usage when available. A turn aggregator publishes

Evaluation	Baseline	Candidate
Public-train total	129/129	129/129
Base	50/50	50/50
Hallucination	48/48	48/48
Disambiguation	31/31	31/31
Targeted same IDs	1/5	5/5
Legacy contract	217/219	217/219

Table 1: Development results. The targeted user-simulator trajectories can diverge after the assistant behavior changes, so that row is supporting rather than transcript-identical causal evidence.

Observed metric	Candidate run
Wall time, 129 tasks	382.85 s
Mean task A2A time	79.9 ms
Median task A2A time	67.6 ms
P95 task A2A time	176.0 ms
Maximum task A2A time	229.0 ms
Mean A2A turns per task	4.15
P95 A2A turns per task	9
Run errors / OOM kills	0 / 0

Table 2: Measurements extracted from the final public-train evaluator artifact. A2A time is the evaluator-reported agent exchange time, not end-to-end task latency or isolated Cerebras latency.

prompt_tokens, completion_tokens, thinking_tokens, call count, pass count, and average LLM latency in A2A turn_metrics. Raw provider responses and private state are excluded.

6 Validation Protocol

We separated promotion evidence from adaptive development. The immutable baseline is commit 185109f; the candidate is its direct child, b924659. A 16-case blind package was frozen before candidate execution and retained SHA-256 80d7e254d3ce...f24a61c. Promotion required no regression on the local full set, fixes for critical blind false-completion cases, focused mechanism tests, source hygiene, Docker execution, and no candidate-only legacy contract failure.

The final candidate full run completed in 382.85 seconds with client exit zero and no OOM-killed container. All 129 selected tasks passed in one trial. This run reports no agent-side LLM tokens because its selected trajectories remained on deterministic paths; it therefore does not estimate hidden-set token cost. Provider-backed blind replay exercised the Cerebras path and obtained four valid candidates that all selected the required destination-replacement action.

Table 2 distinguishes agent exchange latency from total wall time. The latter also includes the simulated user, tool execution, evaluator work, container scheduling, and concurrent task orchestration. We retain both values because using wall time alone would incorrectly attribute external evaluation cost to the submitted agent. The digest-pinned image is Linux AMD64, runs as a non-root user, and exposes the same

health and A2A interface used by the starter kit.

7 Hardening Results and Ablations

The candidate passed 88/88 normally collectable tests, 8/8 focused hardening probes, 19/19 sentinel and hygiene tests, and 49/49 final-image mechanism and red-team tests. In the recovered historical runtime, baseline and candidate both pass 217/219 legacy response-contract tests. The two shared failures cover no-toll route-selection follow-ups and predate this branch; we make no claim to have fixed them.

Two frozen-input blind cases isolate the principal safety gain. In an argument-keyed navigation case, the baseline incorrectly returned RESPOND-DONE using an earlier destination replacement. The candidate rejected that stale evidence; live replay selected the exact replacement tool with the new location identifier. In a schema-evolution email case, the baseline reused an old email success, while the candidate deferred because the current contact-identifier requirement was unresolved.

The runtime exposes ablation switches for single-pass mode, planner-only mode, fixed rollout count, hard-verifier removal, minority-veto removal, policy-graph removal, and evidence-ledger removal. Our strongest available A/B is the full baseline-to-candidate comparison above: the candidate retains 129/129 while repairing both critical blind false completions and all five targeted tasks. We do not report invented Pass³ or token numbers for configurations not run.

7.1 Failure-Oriented Guard Map

The hardening work was organized by failure mechanism rather than by generated task. Stale completion is addressed by latest-request temporal bounds; wrong-object completion by argument provenance; route-versus-local POI reuse by source and search-argument scope; tool schema drift by runtime schema validation; malformed sample dominance by validity filtering and canonical consensus; and unavailable capability hallucination by the policy graph plus explicit defer behavior. This mapping gives each fix a falsifiable invariant. For example, the navigation invariant is not “handle a particular city” but “a current destination mutation requires a successful current call whose destination argument matches the requested location.”

The same discipline governed language parsing. Added patterns cover structural classes such as reversed route endpoints and negated navigation setup. They do not branch on task IDs, generated entity names, or full challenge utterances. After implementation, a literal scan and frozen blind replay were used to check that the mechanism generalized beyond the triggering transcript.

8 Limitations

The public-train run uses one trial, whereas official ranking uses three independent trials. A perfect local Pass¹ result cannot establish hidden Pass³. Deterministic fast paths also mean that the observed full-run wall time combines evaluator, simulator, Docker, and A2A overhead rather than isolating provider latency. Hidden requests may activate more LLM rollouts and therefore different token and latency profiles.

The rule-based controller is intentionally broad but necessarily incomplete. New schemas, unusual paraphrases, and interactions among otherwise correct obligations remain possible failure sources. To limit overfitting, we stopped adaptive test generation after promotion and retained the prior digest as an immutable rollback artifact.

9 Reproducibility and Deployment

The promoted runtime is commit b924659, tagged candidate-challenge-pareto-v4-20260717.

The submitted artifact is the public `lcthej/car-bench-episco-guard` GHCR package, pinned by the full digest in the scenario. An independent pull resolved that digest to image ID `97e6ef...a43057` on `linux/amd64`; neither reference depends on a mutable tag.

The scenario exposes provider base URL, service tier, model identifiers, reasoning effort, completion caps, concurrency, adaptive rollout counts, retry bounds, and all ablation switches as environment variables. It contains no secret value. The official evaluator image is unchanged, the hidden split is selected, all task counts are -1 , and the evaluator supplies its own secret. This makes the execution configuration auditable without embedding credentials or relying on a mutable image tag.

9.1 Compute-Aware Interpretation

The harness uses speed in two complementary ways. Deterministic fast paths make simple grounded actions cheap: they bypass provider calls when the evidence and policy state determine a unique answer. Ambiguous or high-risk steps instead spend the saved budget on parallel independent candidates. Because the rollout fan-out is concurrent, its latency is closer to the slowest member than to the sum of all candidate latencies. Canonical grouping then converts that extra inference into action-level consensus.

Adaptive rollout count prevents best-of-eight sampling from becoming the default cost for every turn. Low-risk fallbacks begin at two samples; medium and high risk use four or eight. Deterministic risk features include unresolved obligations, potential side effects, schema uncertainty, and conflict among candidate action kinds. The cap remains configurable, allowing organizers to audit the submitted defaults or lower concurrency without changing code.

Sequential depth is bounded independently of parallel width. Fast paths use no LLM; generative paths use independent rollout branches whose initial attempt and at most two retries give a maximum depth of three. All post-rollout stages are deterministic. Failed provider calls are counted in turn metrics even when they contribute no token usage. Retry count and backoff are bounded, so rate limiting cannot create an unbounded reasoning loop.

9.2 Component Evidence

The evidence in Table 3 supports promotion, but not a claim that every component is individually optimal. In particular, the full run does not isolate the marginal benefit of each rollout count because its selected trajectories used deterministic paths. The runtime switches preserve a clean future ablation

Mechanism	Available evidence
Temporal and argument scope	Two critical frozen blind false completions repaired
Canonical consensus	Malformed candidates excluded; four valid live samples agreed
Policy/evidence fast paths	129/129 with zero agent LLM tokens in the selected run
Generic parser hardening	Same-day targeted evaluation improved from 1/5 to 5/5
Image and A2A boundary	49/49 mechanism/red-team tests; digest pull on Linux AMD64

Table 3: Each reported mechanism is tied to an observed development artifact; none is justified by hidden-set feedback.

route, while the competition candidate remains fixed to the configuration that passed all promotion gates.

Operationally, each conversation is isolated by context identifier. Returned tool results are consumed only through evaluator-provided A2A messages, and pending call provenance is cleared after association. Malformed provider output is rejected by parsing and verification; it is not sent to a repair LLM. These properties make a container restart or a partial provider failure fail closed rather than produce an unsupported completion claim.

10 Conclusion

EpiSCo-Guard converts fast parallel inference into reliability through an evidence-first control plane. Cerebras rollouts propose diverse actions; deterministic provenance, policy, schema, and consensus checks decide which action is safe to expose. The resulting challenge candidate preserves the full local score while correcting stale-evidence failures that the submitted baseline missed. Its remaining uncertainty is explicit: only the organizer’s hidden three-trial evaluation can determine final CAR-bench performance.

References

- [A2A Protocol Working Group, 2025] A2A Protocol Working Group. Agent2agent (A2A) protocol specification, version 0.3.0, 2025.
- [Cerebras Systems, 2026] Cerebras Systems. Cerebras inference: Public models, 2026. Accessed 2026-07-22.
- [Kirmayr *et al.*, 2026] Johannes Kirmayr, Lukas Stappen, and Elisabeth André. CAR-bench: Evaluating the consistency and limit-awareness of LLM agents under real-world uncertainty. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics*, 2026.
- [OpenAI, 2025] OpenAI. gpt-oss-120b and gpt-oss-20b model card, 2025.
- [Wang *et al.*, 2023] Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc V. Le, Ed H. Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou. Self-consistency improves chain of thought reasoning in language models. *International Conference on Learning Representations*, 2023.
- [Yao *et al.*, 2023] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. *International Conference on Learning Representations*, 2023.